

《数字信号处理》课程设计 **DTMF**信号的检测与识别

无96 吴昊宸 2019011126

一、实验目标

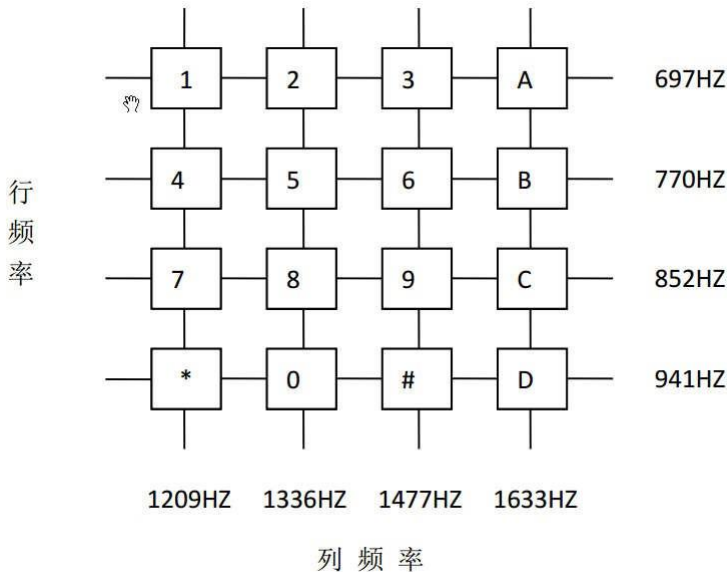
- 利用中文教材5.10.1节介绍的第二种数字正弦振荡器设计方法，编写DTMF信号产生的MATLAN程序(输入任意按键可以产生其对应的DTMF数字信号波形输出)。
- 利用FFT和Goertzel算法对附件一中给定短音频文件中的DTMF信号进行检测和识别
- 对附件二中给定长音频文件中的DTMF信号进行检测与识别

二、实验原理与说明

在这个部分，我将分任务的对我的设计思路进行阐述

• **DTMF**信号的生成

由实验指导书我们可以知道，DTMF实际上就是将按键映射成2个对应的频率，之后生成两个频率的三角函数信号并将其叠加而已。在知道按键-频率对应关系(下图)的情况下，核心问题就是如何生成正弦/余弦信号。



根据实验指导书的指示，我在中文教材的5.10.1节中找到了答案。利用三角函数和差的性质，构造一个特殊的递归系统如下：

$$\begin{aligned}
y_c[-1] &= \cos(\omega_0) \\
y_s[-1] &= -\sin(\omega_0) \\
y_c[n] &= (\cos\omega_0)y_c[n-1] - (\sin\omega_0)y_s[n-1] \\
y_s[n] &= (\sin\omega_0)y_c[n-1] + (\cos\omega_0)y_s[n-1]
\end{aligned}$$

这样， $y_c[n]$ 和 $y_s[n]$ 就能输出 $\cos\omega_0 n$ 和 $\sin\omega_0 n$ 的正余弦信号。

已知采样频率 $f_s = 8k\text{Hz}$ ，在需要生成的三角函数信号为 f 的情况下，我们只需要取 $\omega_0 = \frac{f}{f_s}$ ，带入上面的递推系统中进行计算，就能得到相应的函数采样值了。

核心代码如下(这是利用这个迭代系统生成频率为 f_1 ,采样频率为 f_s 的正弦SinSeq1/余弦信号CosSeq1的代码):

```

w1 = 2 * pi * f1 / fs;
CosSeq1 = zeros(1, fs + 1); SinSeq1 = zeros(1, fs + 1); %给-1点留个位置
CosSeq1(1)=cos(w1); SinSeq1(1)=-sin(w1);
for i=2:8001
    CosSeq1(i)=cos(w1)*CosSeq1(i-1)-sin(w1)*SinSeq1(i-1);
    SinSeq1(i)=sin(w1)*CosSeq1(i-1)+cos(w1)*SinSeq1(i-1);
end

```

从按键-频率映射中取出 f_1, f_2 ,利用这个模块生成SinSeq1和SinSeq2，然后将两者取平均，就可以得到所要求的DTMF信号的输出，从而完成这一部分的任务。

具体代码见**DTMFGenerator.m**

- 短音频文件的**DTMF**信号检测

1. FFT检测法

因为DTMF信号的本质就是两个正弦信号的叠加，因此我们对收到的DTMF信号直接进行FFT，在DTMF映射的指定频率上提取频率分量，然后再进行比较，就能筛选出原本DTMF信号中的 f_1, f_2 ，从而得到被编码的按键。

因此代码的逻辑就非常清楚了。首先读入音频文件，然后进行FFT变换，再在DTMF配置的频率点上提取幅度分量进行比较，取最大值恢复原本的按键。同时，我为了防止频率分量的波动可能导致的错误，在相应DTMF的频率点周围取了平均数。

核心代码如下(经过预处理,我发现所有音频信号的长度均<2000,因此我取FFT变换点数为2000,在yFFT的序号和原本频率之间建立一个比较清楚的4倍关系):

```

for i = 1:len
    [y, Fs] = audioread(FileNameList(i));
    % 通过打印length(y)我们可以发现,10个文件中的音频信号数组长度都在2000以内
    % 且音频信号都是-1~1之间的整数
    % 我们利用FFT的方法进行频谱分析
    yFFT = abs(fft(y, N)); %得到2000点FFT数据,其中第i个点代表f=4*i的频率分量
    stem(yFFT);
    SampleAmp = (yFFT(index) + yFFT(index - 1) + yFFT(index) + 1) / 3; %做个滑动平
    均防止有点偏差
    [rowmax, row] = max(SampleAmp(1:4)); %前四个判断行号
    [colmax, col] = max(SampleAmp(5:8)); %后四个判断列号
    output = [output, KeyMap(row, col)];
end

```

运行这段代码，得到的结果为'**5169873402**'，这就是这**10**个音频文件所分别代表的数字。

命令行窗口		工作区
名称	值	
col	2	
colmax	19.7212	
FileNameList	1x10 string	
freq	[697,770,852,9...	
fs	8000	
Fs	8000	
i	10	
index	[174,192,213,2...	
KeyMap	4x4 char	
len	10	
N	2000	
output	'5169873402'	
row	1	
rowmax	95.5763	
SampleAmp	[95.5763;11.15...	
y	1944x1 double	
yFFT	2000x1 double	

具体代码见**SingleDTMFDetector_FFT.m**

2. Goertzel检测法

根据实验指导书上的思路，Goertzel检测法的思路是用特定频点上FFT系数的计算来替代整个FFT操作。而特定频点 k 上，FFT系数 $X[k]$ 的计算可以利用下述方式实现：

$$\begin{aligned}v_k[n] &= 2\cos(\omega_k)v_k[n-1] + v_k[n-2] + x[n] \\y_k[n] &= v_k[n] - W_N^k v_k[n-1] \\X[k] &= y_k[N]\end{aligned}$$

其中 $\omega_k = \frac{2\pi k}{N}$, $W_N = e^{-j\frac{2\pi}{N}}$, $v_k[n]$ 是初始状态为0的状态数组， N 是FFT点数。

将Goertzel算法运行8次，我们就可以知道输入信号在DTMF指定频率点上的幅度分量大小，之后和FFT算法一样进行比较判别即可。核心代码如下(注意因为我们没有用MATLAB给定的集成fft函数，因此输入的序列 $\mathbf{x}$ 需要补零直到长度为 N)：

```
for i = 1:len
    [x, Fs] = audioread(FilenameList(i));
    x = [x; zeros(N - length(x), 1)];
    % 接下来利用Goertzel算法计算特定频点上的FFT系数(对每个频率都计算一次)
    for k = 1:8
        v(1) = x(1); %初始状态v[1]=x[1]
        v(2) = 2 * Coswk(k) * v(1) + x(2); %初始状态v[2]

        for n = 3:N
            v(n) = 2 * Coswk(k) * v(n-1) - v(n-2) + x(n);
        end

        X(k) = abs(v(N) - exp(-1j * 2 * pi * k / N) * v(N-1));
    end

    % 计算出了特定频率的值以后就可以进行判定
    [rowmax, row] = max(X(1:4));
    [colmax, col] = max(X(5:8));
    output = [output, KeyMap(row, col)];
end
```

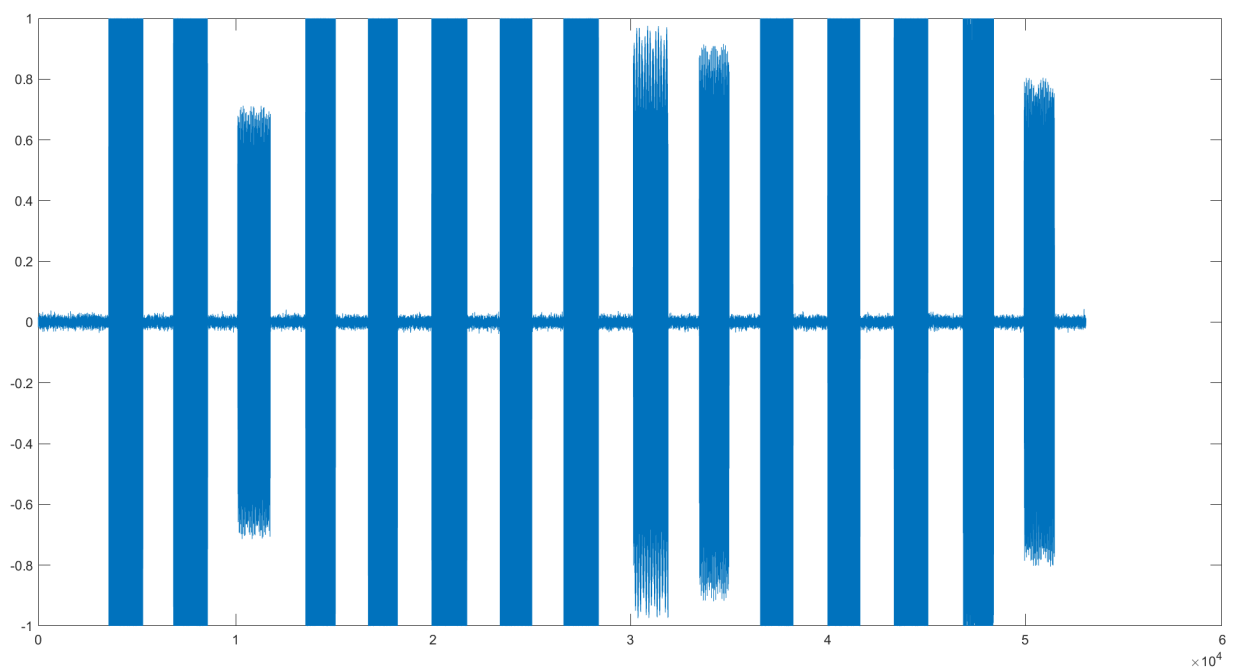
执行整段代码，得到结果为'**5169873402**'，和FFT算法所得到的结果保持一致，变相确认了两个算法的正确性

col	2
colmax	401.0630
Coswk	[0.8543,0.8235...
str FilenameList	1x10 string
freq	[697,770,852,9...
fs	8000
Fs	8000
i	10
index	[174,192,213,2...
k	8
ch KeyMap	4x4 char
len	10
n	2000
N	2000
ch output	'5169873402'
row	1
rowmax	307.6667
v	1x2001 double
x	2000x1 double
X	[307.6667,8.49...

具体代码见文件 **SingleDTMFDetector_Goertzel.m**

- 长音频文件的**DTMF**信号检测

当步入长音频文件DTMF信号检测的领域的时候，核心问题就从单音节的DTMF解码变成了如何分辨并且切分不同的音节。我们首先读入并且显示data.wav文件：



可以看到data.wav文件中有着15段非常明显的音节。一个理想的算法应该能将这15段音节的开始/结束点全部标记出来。

在这里我采用这种办法：取一个较短的窗长观察窗内的信号功率和，若这个和从阈值之下跃升到了阈值之上，则判定为音节开始；如果从阈值之上降低到了阈值之下，则判定为音节结束。在阈值上/下不变的情况下，音节状态也不变化。

这种分段方法的核心代码如下：

```
function [head, tail] = AudioSeperate(input)
    % 音频分割函数,能将input中固定时间内功率超过阈值的区间截取出来

    WindowLen = 10; %窗长为10
    DataLen = length(input); %数据长度
    head = []; tail = []; %初始化

    threshold = 0.1; %经过尝试得到的阈值
    isKey = 0; %是否是按键指示变量

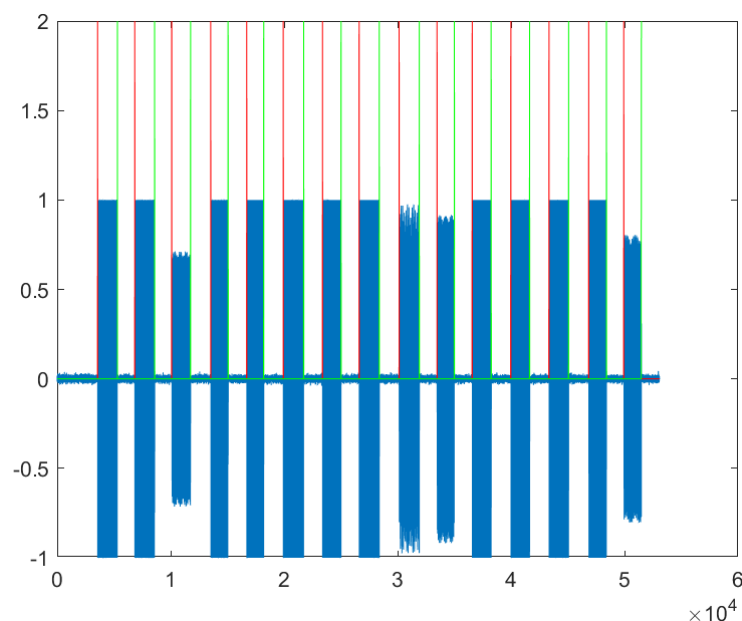
    for i = 1:DataLen - WindowLen + 1 %遍历所有长度为WindowLen的区间
        power = sum(input(i:i + WindowLen - 1).^2); %计算功率

        if (power > threshold && isKey == 0) %超过阈值且没检测到为key
            head = [head, i]; %更新起始点
            isKey = 1;
        end

        if (power <= threshold && isKey == 1) %没超过阈值且正在Key中
            tail = [tail, i];
            isKey = 0; %更新结束点
        end
    end
end
```

这个函数所返回的head和tail数组就是input的音节起始和结束的坐标。而阈值threshold和窗长WindowLen是我根据调试而确定的。

运行这个算法，并且在图上标记处head和tail的位置，结果如下：



可见我所编写的AudioSeperate函数能够比较完美的对data.wav中的音节进行辨别与分割。

之后再对每个音节进行DTMF按键的判别，利用FFT或者Goertzel算法都可以。在文件中我两个算法都予以了执行，最终得到的结果为‘20589113209’，无论是FFT还是Goertzel算法的结果都是这个，这变相证明了我所编写的代码的正确性。

```
>> MultipleDTMFDetector
205891132094649
205891132094649
```

具体代码见MultipleDTMFDetector.m文件

三、实验心得

在本次课程设计中，我应用第一次课程设计所涉及的FFT算法来解决实际的DTMF信号识别的问题。同时，我还学习了FFT算法的拓展——Goertzel算法，对DTMF信号识别与分离的功能进行了优化。不仅如此，在任务三中的DTMF音节的分离部分，我也融会贯通，应用了模式识别中的相应算法来解决实际问题。这都大大锻炼了我的思考能力，令我收获颇多。

而在编程技巧方面，我体会到了在信号处理方面上，MATLAB语言相对于简单的C语言的优越性。在MATLAB中不需要编写冗杂的复数类，也不需要自己实现并优化FFT算法。排除这些小事的干扰，我可以将精力完全集中在DTMF识别的任务之上。

同时我也思考了我所编写的识别方法的不足之处。感觉上这种方法对于噪音，以及DTMF音节之间的串扰的容忍性并不是很高，也许是一个以后可以进行探究提升的方向。

总而言之，我在这次课程设计中，无论是在课程知识，还是在代码经验上都受益匪浅。谢谢助教以及老师为这次课程设计所作出的付出！